

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an **OCR system** that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect **facial expressions (happy, sad, neutral)** with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

6. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

7. Smart Attendance System using Face Recognition

A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

8. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

9. Document Scanner Application

A mobile app must:

- A. Capture document images
- B. Enhance readability
- C. Output clean scanned images

Design the OpenCV pipeline and justify preprocessing steps used for improving document quality.

10. Vehicle Detection in Parking System

A parking system must:

- A. Detect cars entering/exiting
- B. Track movement
- C. Update available slots

Design the pipeline and justify how your approach handles multiple vehicles and avoids counting errors.

11. Gesture Recognition System

A system must:

- A. Capture hand gestures from a webcam
- B. Identify simple gestures
- C. Respond in real time

Design an OpenCV pipeline and justify feature extraction and classification methods.

12. Video Frame Quality Enhancement

A system must:

- A. Improve video quality in real time
- B. Reduce noise and enhance contrast

Design the pipeline and justify enhancement techniques used.

13. Multi-Face Tracking System

A system must:

- A. Detect multiple faces
- B. Track them across frames
- C. Maintain identity

Design the pipeline and justify tracking strategy.

14. OCR for License Plate Recognition

A system must:

- A. Detect license plates
- B. Extract text
- C. Work under varying lighting

Design the OpenCV pipeline and justify preprocessing and segmentation methods.

15. Real-Time Edge Detection System

A system must:

- A. Capture video
- B. Detect edges in real time
- C. Display results efficiently

Design the pipeline and justify optimization strategies.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

		tools			
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

SIMATS ENGINEERING

Assessment Tool 1: Constraint-Based Problem Solving — Answer Sheet

Course Name: Computer Vision | Course Code: ITA05

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Name: VIGNESHWAR S V Reg. No: 192321135

Questions Answered: 2, 3, 7, 9, 15

2. Automated Video Surveillance with Alert System

Objective: Detect motion and unusual activities from standard CCTV feeds, generate real-time alerts, and minimize false positives.

Pipeline: CCTV Video Feed → Frame Acquisition → Noise Reduction → Background Subtraction → Motion Detection → Feature/Object Detection → Activity Analysis → False Positive Filtering → Alert Generation

Method

1. Video acquisition: OpenCV VideoCapture() reads CCTV frames.
2. Preprocessing: Convert to grayscale and apply Gaussian blur to reduce sensor noise.
3. Motion detection: MOG2 background subtraction extracts foreground motion.
4. Feature detection: Contours identify moving regions; small regions are ignored.
5. Unusual activity: Persistent or sufficiently large motion can be treated as suspicious.
6. Alert: Generate an alert when motion exceeds a defined threshold.

Handling Noise and Dynamic Backgrounds

Gaussian filtering and morphological operations remove small noise. Background subtraction separates moving foreground from the background. Area filtering and persistence checks reduce false alarms caused by shadows, trees, or brief changes.

Real-Time Optimization

Resize frames, use lightweight OpenCV operations, and avoid unnecessary processing.

Python / OpenCV Code

```
import cv2
cap = cv2.VideoCapture(0)
bg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=40,
    detectShadows=True)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 480))
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (5, 5), 0)
    mask = bg.apply(gray)
    _, mask = cv2.threshold(mask, 200, 255, cv2.THRESH_BINARY)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.dilate(mask, kernel, iterations=2)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE)
    motion_detected = False
    for contour in contours:
        if cv2.contourArea(contour) > 1500:
            x, y, w, h = cv2.boundingRect(contour)
```

```

        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        motion_detected = True
    if motion_detected:
        cv2.putText(frame, "ALERT: MOTION DETECTED", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
    cv2.imshow("Surveillance", frame)
    cv2.imshow("Motion Mask", mask)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()

```

Conclusion: The system combines preprocessing, background subtraction, motion analysis and alert generation to provide an efficient real-time surveillance solution.

3. OCR System for Low-Quality Documents

Objective: Extract text from low-resolution, noisy scanned images while handling varying fonts and illumination efficiently.

Pipeline: Input Image → Resize → Grayscale → Noise Removal → Contrast Enhancement → Adaptive Thresholding → Morphological Processing → Text Segmentation → OCR → Extracted Text

Processing Stages

Resize improves small characters; grayscale simplifies processing; median filtering removes noise; CLAHE improves local contrast; adaptive thresholding handles uneven illumination; morphology removes small noise and strengthens characters; OCR converts the processed image into text.

Stage	Purpose
Resize	Improves visibility of small characters
Grayscale	Simplifies image processing
Median filtering	Removes noise
CLAHE	Improves local contrast and handles illumination
Adaptive thresholding	Separates text from uneven background
Morphology	Removes noise and improves character shapes
OCR	Converts image text into editable text

Python / OpenCV Code

```

import cv2
import pytesseract
img = cv2.imread("document.jpg")
img = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.medianBlur(gray, 3)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(gray)
binary = cv2.adaptiveThreshold(
    enhanced, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 31, 10)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
clean = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
text = pytesseract.image_to_string(clean)
print("Extracted Text:")
print(text)
cv2.imshow("Processed", clean)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Conclusion: The pipeline improves OCR recognition through resolution enhancement, noise removal, contrast enhancement, adaptive segmentation and morphological processing.

7. Smart Attendance System Using Face Recognition

Objective: Capture faces from a webcam, recognize registered users, and automatically mark attendance while minimizing false recognition.

Pipeline: Webcam → Frame Capture → Face Detection → Face Preprocessing → Face Recognition → Confidence Verification → Registered User? → Attendance Marking

Processing

Capture video using OpenCV; detect faces using a Haar Cascade; extract and preprocess face regions; compare against registered faces; apply a confidence threshold; record attendance only for reliable matches.

False-Recognition Prevention

Use multiple training images, a suitable recognition threshold, reject low-confidence matches as Unknown, use good-quality face samples, prevent duplicate attendance, and add liveness detection for higher-security systems.

Python / OpenCV Code

```
import cv2
import os
import numpy as np
from datetime import datetime

face_detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
recognizer = cv2.face.LBPHFaceRecognizer_create()
faces, labels, names = [], [], {}
data_path = "faces"
label_id = 0
for person in os.listdir(data_path):
    person_path = os.path.join(data_path, person)
    if not os.path.isdir(person_path):
        continue
    names[label_id] = person
    for file in os.listdir(person_path):
        image = cv2.imread(os.path.join(person_path, file),
                           cv2.IMREAD_GRAYSCALE)
        if image is None:
            continue
        detected = face_detector.detectMultiScale(image, 1.1, 5)
        for (x, y, w, h) in detected:
            faces.append(image[y:y+h, x:x+w])
            labels.append(label_id)
        label_id += 1
recognizer.train(faces, np.array(labels))

cap = cv2.VideoCapture(0)
attendance = set()
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    detected = face_detector.detectMultiScale(gray, 1.2, 5)
    for (x, y, w, h) in detected:
        face = gray[y:y+h, x:x+w]
        label, confidence = recognizer.predict(face)
        if confidence < 70:
            name = names[label]
            if name not in attendance:
```

```

        attendance.add(name)
        with open("attendance.csv", "a") as f:
            time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            f.write(name + "," + time + "\n")
    else:
        name = "Unknown"
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, name, (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
        cv2.imshow("Smart Attendance", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    cap.release()
    cv2.destroyAllWindows()

```

Conclusion: The system automates attendance using face detection, recognition, confidence verification and attendance logging.

9. Document Scanner Application

Objective: Capture a document, remove unwanted background, correct perspective, enhance readability and output a clean scanned image.

Pipeline: Camera Image → Grayscale → Blur/Noise Removal → Edge Detection → Contour Detection → Document Boundary → Perspective Transformation → Contrast/Thresholding → Clean Scan

Justification

Grayscale simplifies processing; Gaussian blur reduces camera noise; Canny detects document edges; contour detection identifies the page; perspective transformation corrects tilt; adaptive thresholding produces a readable document.

Python / OpenCV Code

```

import cv2
import numpy as np

image = cv2.imread("document.jpg")
ratio = image.shape[0] / 500.0
orig = image.copy()
image = cv2.resize(image, (int(image.shape[1] / ratio), 500))
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 75, 200)
contours, _ = cv2.findContours(edges, cv2.RETR_LIST,
                               cv2.CHAIN_APPROX_SIMPLE)
contours = sorted(contours, key=cv2.contourArea, reverse=True)

document = None
for contour in contours:
    perimeter = cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, 0.02 * perimeter, True)
    if len(approx) == 4:
        document = approx
        break

if document is not None:
    points = document.reshape(4, 2)
    s = points.sum(axis=1)
    diff = np.diff(points, axis=1)
    top_left = points[np.argmin(s)]
    bottom_right = points[np.argmax(s)]
    top_right = points[np.argmax(diff)]
    bottom_left = points[np.argmin(diff)]
    ordered = np.array([top_left, top_right, bottom_right, bottom_left],
                      dtype="float32")
    width, height = 600, 800
    destination = np.array([[0, 0], [width-1, 0],
                           [width-1, height-1], [0, height-1]],
                          dtype="float32")

```

```

matrix = cv2.getPerspectiveTransform(ordered, destination)
scanned = cv2.warpPerspective(orig, matrix, (width, height))
scanned_gray = cv2.cvtColor(scanned, cv2.COLOR_BGR2GRAY)
scanned_gray = cv2.GaussianBlur(scanned_gray, (3, 3), 0)
clean = cv2.adaptiveThreshold(
    scanned_gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 21, 10)
cv2.imwrite("scanned_document.png", clean)
cv2.imshow("Scanned Document", clean)

cv2.imshow("Original", orig)
cv2.imshow("Edges", edges)
cv2.waitKey(0)

```

Conclusion: The pipeline produces a perspective-corrected and enhanced digital document similar to a basic mobile scanning application.

15. Real-Time Edge Detection System

Objective: Capture video, detect edges continuously, and display the results efficiently in real time.

Pipeline: Webcam → Frame Capture → Resize → Grayscale → Gaussian Blur → Canny Edge Detection → Display

Optimization Strategies

Resize frames to lower resolution; convert directly to grayscale; use Gaussian filtering to reduce noise; use optimized OpenCV operations; avoid unnecessary image conversions; process only a region of interest when possible.

Python / OpenCV Code

```

import cv2
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 480))
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blur, 50, 150)
    cv2.imshow("Original Video", frame)
    cv2.imshow("Real-Time Edges", edges)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()

```

Why Canny?

Canny provides effective edge detection with noise suppression, gradient analysis, non-maximum suppression and thin edge output.

Conclusion: The proposed system provides efficient real-time edge detection using grayscale conversion, Gaussian smoothing and Canny detection. Lower resolution and minimal processing help maintain real-time performance.

Code of Conduct

I, **VIGNESHWAR S V (Reg. No: 192321135)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____